

Report: Secure Network Application Development Using TCP

1. Objective

The primary objective of this project is to enhance a GUI-based multi-client TCP application by designing and integrating practical, application-layer security mechanisms. These features target key elements of the CIA Triad (Confidentiality, Integrity, and Availability) by implementing secure authentication, password hashing, session timeout protocols, input verification, duplicate login prevention, and rate-limiting brute-force defenses. Validation of these technical controls is verified through live packet-level analysis via Wireshark.

2. Security Features Implemented

- **User Authentication & Protocol Handling:** Implemented a custom AUTH `<username> <password>` command pattern that forces clients to validate identities before accessing any workspace messaging systems.
- **Secure Password Storage:** Plaintext credentials are never handled directly by storage systems. The server utilizes the SHA-256 cryptographic algorithm (`hashlib.sha256()`) to convert passwords into one-way hashes, which are safely managed inside a localized `users.json` file.
- **Duplicate Login Prevention:** To preserve session integrity, the server maintains an active in-memory user registry (`active_usernames`). If an identical username attempts a concurrent login, the incoming socket connection is instantly rejected.
- **Strict Input Validation:** The application actively scrubs all inputs to eliminate malicious formatting strings, drops null/empty fields, enforces maximum payload thresholds (1024 bytes) to mitigate buffer exploit risks, and rejects unrecognized user commands.
- **Failed Login Protection (Brute-Force Lockout):** Tracks connection activity by tracking source IP addresses. If an IP registers three sequential authentication failures (`MAX_FAILED_ATTEMPTS = 3`), the server moves the IP into a temporary blacklist (`blocked_ips`) for a 30-second cooling period.
- **Session Management & Secure Logging:** Includes explicit LOGOUT command recognition, drops inactive sockets automatically via inactivity timers (`TIMEOUT_LIMIT = 300.0`), and records administrative auditing actions to `security_log.txt` without logging sensitive password fields.

3. System Architecture

The lab environment utilizes Mininet to virtualize an isolated single network switch topology (s1) interconnecting five unique host nodes (h1 through h5).

- **Host 1 (h1 / 10.0.0.1):** Acts as the central, multi-threaded TCP Server managing the application logic, users.json database, and the reentrant logging thread.
- **Host 2 (h2 / 10.0.0.2):** Acts as the active target client platform executing the client_gui.py user application interface.

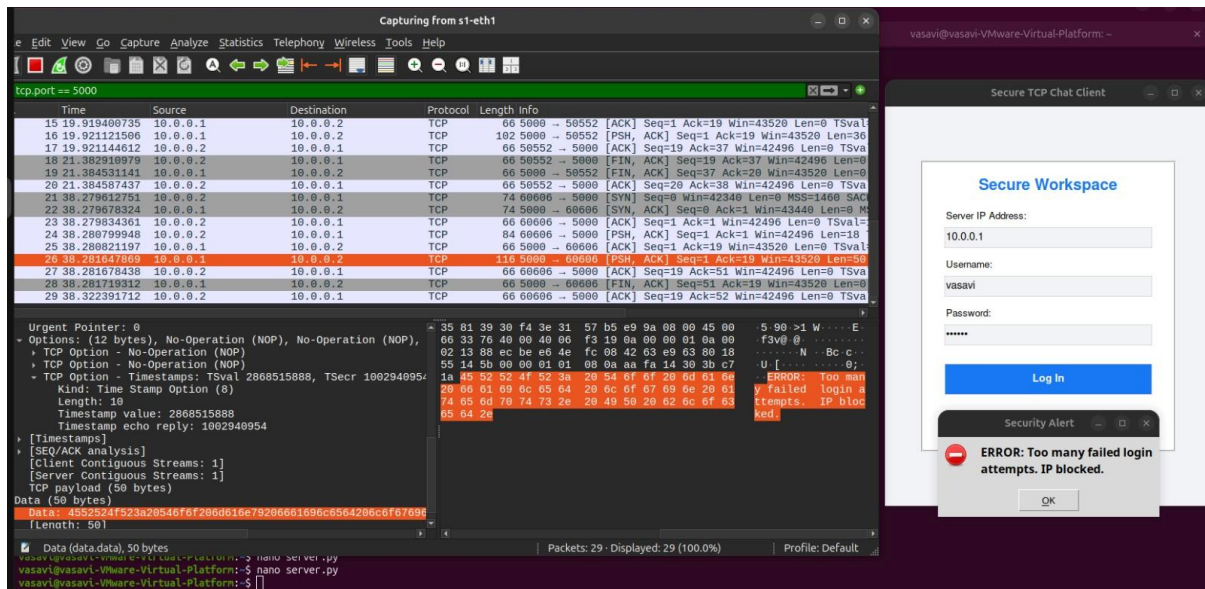
4. Implementation Details & Testing

During implementation, two critical architectural hurdles were identified and successfully resolved:

1. **Reentrant Logging Deadlock:** Testing revealed that a standard mutex lock (threading.Lock()) caused the server thread to freeze indefinitely on the 3rd failed attempt because record_failed_login() tried to call log_event() while still holding the lock. This was resolved by upgrading the thread state to a Reentrant Lock (threading.RLock()).
2. **Socket Buffer Race Condition:** The server was severing socket connections too quickly for the physical interface buffers to catch the text data payload. This was mitigated by introducing client_socket.shutdown(socket.SHUT_WR), forcing the operating system kernel to completely flush outbound buffers down the network wire before final socket closure.

5. Wireshark Verification

To confirm compliance with Task 7, a network traffic capture was conducted on the Mininet switch interface using the tracking filter tcp.port == 5000.



Traffic Analysis:

- **Packet Structure:** Packet No. 26 originates from the server host (10.0.0.1) routed to the client host (10.0.0.2).

- **Payload Verification:** The packet length measures exactly 116 bytes, which explicitly contains the raw ASCII data payload: ERROR: Too many failed login attempts. IP blocked.
- **TCP Flow Lifecycle:** The network capture proves that the server pushes this warning string across the wire first, receives a client acknowledgment (ACK), and then immediately initiates a graceful four-way TCP teardown handshake (FIN, ACK), validating the robust performance of our brute-force application protection.

6. Conclusion

The implementation successfully meets all specified technical objectives. By deploying rate-limiting parameters, secure password hashing, and active validation layers directly into the application space, the system maintains robust resilience against script-driven dictionary attacks. Packet analysis verifies that the server successfully blocks threats while communicating clearly with the endpoint before dropping connections.